

1. Chosen Language and Technical Path

Python 3 with a layered MVC structure. Reasons: standard library is rich enough that AstraNotes can be built with minimal third-party dependencies (which directly addresses an auditability concern in the Charter), cryptography is a well-vetted package for the SecureNote encryption story, and clean module boundaries are easy to enforce without ceremony.

2. Weaker Architecture Prompt

Design a simple storage system for AstraNotes so users can save notes.

Why this prompt is weak

It under-specifies the audience, the architectural style, the non-functional concerns (privacy, performance, extensibility), and the realization context. The AI's first response gave a single-class outline with `save_note(text)` writing to a flat `.txt` file, no encryption boundary, no error handling, and no path for the Voice or Secure plugins required by the Charter. It did not surface module boundaries.

3. Stronger Architecture Prompt

Act as a senior software architect. Design a modular storage architecture for AstraNotes in Python 3, following the MVC pattern from the Project Charter. AstraNotes is a local-first, cross-platform note-taking tool that must support Text, Voice, and Secure notes via a plugin interface, markdown rendering, version history, and a future Cloud Sync service. Think step by step. Define (a) the Note domain model and SecureNote subtype, (b) a Repository interface that abstracts persistence, (c) where encryption, validation, and version-history responsibilities live, (d) how a Plugin can extend note types without rewriting the storage layer, and (e) how the design stays testable. Avoid unverified third-party dependencies and call out any assumption you make.

Why this prompt is stronger

It names the architectural style, the Charter constraints (plugins, markdown, version history, Cloud Sync), the language, and the reasoning method. It forces the AI to reason about boundaries instead of generating boilerplate, and asks it to surface assumptions, which makes it easier to detect hallucination.

4. Excerpt — Stronger AI Response

Domain layer (Model). Note is the base entity (id: UUID, title: str, body: str, created_at, updated_at, kind: NoteKind). SecureNote extends Note with `passphrase_hash` and an `encrypted_body`: bytes; the plaintext body is never stored on disk.

Repository (Model). NoteRepository is an abstract interface (save, get, list, delete, history). The default implementation SQLiteNoteRepository persists to a local SQLite file via the sqlite3 stdlib module. Swapping in a RemoteNoteRepository later is how Cloud Sync is added without touching callers.

Services (Controller). EncryptionService (wraps cryptography.fernet for SecureNote bodies), ValidationService (rejects invalid state before save), VersionHistoryService (snapshots before update), NoteService (orchestrates the workflow and is the only entry point for the View), PluginManager (registers NoteKind → Handler so Voice / Text / Secure can be added without changing core logic).

View. A thin CLI/desktop view layer that renders markdown and routes commands to NoteService. The View never imports persistence or encryption modules directly. Assumptions surfaced: SQLite is acceptable for the 10k-note responsiveness target; cryptography is the only external package required for Phase 1; plugin loading is in-process (no sandboxing in scope).

5. Verdict

The stronger architecture is more defensible because it makes module boundaries visible, places privacy logic behind a service the View cannot bypass, and keeps Cloud Sync as a future Repository swap rather than a cross-cutting rewrite. It also gives me a backlog-ready vocabulary — Note, NoteRepository, EncryptionService, NoteService, PluginManager — that downstream UML, requirements, and traceability artifacts can reuse without contradicting themselves.